100 HARD INTERVIEW QUESTIONS

Data Scientist Position

Business Stream - COO Function

Band C | Business Intelligence & Data Department

Prepared for: Yash B.

MSc Computer Science

Part I: Role Overview & Analysis

The Scrying Ritual - Intelligence Analysis

A) Role Archetype: Analytics Engineer + Applied Scientist Hybrid

This Band C (Senior) Data Scientist role at Business Stream is NOT a pure ML Engineering position. The JD emphasizes:

- Power BI & Visualization (40%) - Your gap area
- SQL & Data Manipulation (30%) - Your strength
- Prescriptive Analytics & Business Insights (20%) - Adjacent to your ML background
- Data Governance & Utilities Domain (10%) - New territory

B) Company Culture Essence

Dimension	Business Stream Profile
Velocity	"Measure twice, cut once" - regulated utilities move deliberately
Driver	Compliance-driven (OFWAT) + Customer service focused
Stage	Established Enterprise (2000+) - mature processes, governance-heavy
Constraints	Heavy regulation, GDPR, OFWAT reporting, public scrutiny

C) The Gap Between Worlds - Ruthless Assessment

Gap	Severity	Mitigation Strategy
Power BI depth	HIGH	Your Tableau skills transfer ~60%; need DAX mastery proof
Utilities domain	MEDIUM	Retail analytics experience transfers to customer segmentation
Time-series forecasting	MEDIUM	NOAA project shows capability, but utilities have unique patterns
Stakeholder management at scale	MEDIUM	Co-op leadership is good, but enterprise is different
Regulatory compliance mindset	MODERATE	Must demonstrate GDPR/audit trail thinking

D) Competitive Moats to Emphasize

Your Unique Strength	Why It Matters for Business Stream
MLOps + Production Systems	They want to move to prescriptive analytics - you can build the pipelines
Cloud Architecture (AWS/GCP)	Microsoft Fabric is new - your cloud agility is valuable
18 Projects in 90 Days	Demonstrates rapid learning and delivery under pressure
FastAPI + Sub-second Latency	Real-time dashboards and APIs for operational data
Retail Analytics Experience	Customer churn, segmentation, forecasting - directly applicable

Part II: SQL & Data Manipulation (13 Questions)

Q1: Rolling Consumption Anomaly Detection

Scenario: Identify customers whose 3-month consumption exceeds 2.5 standard deviations above their 12-month rolling average.

Skills Tested: Window functions, statistical analysis, time-series

```
WITH MonthlyConsumption AS (
  SELECT CustomerID, DATE_TRUNC('month', ReadingDate) AS Month,
    SUM(Consumption) AS MonthlyTotal
  FROM MeterReadings
  GROUP BY CustomerID, DATE_TRUNC('month', ReadingDate)
),
RollingStats AS (
  SELECT CustomerID, Month, MonthlyTotal,
    AVG(MonthlyTotal) OVER (PARTITION BY CustomerID ORDER BY Month
      ROWS BETWEEN 12 PRECEDING AND 1 PRECEDING) AS RollingAvg,
    STDDEV(MonthlyTotal) OVER (PARTITION BY CustomerID ORDER BY Month
      ROWS BETWEEN 12 PRECEDING AND 1 PRECEDING) AS RollingStdDev
  FROM MonthlyConsumption
)
SELECT CustomerID, Month, MonthlyTotal, RollingAvg, RollingStdDev,
  (MonthlyTotal - RollingAvg) / NULLIF(RollingStdDev, 0) AS ZScore
FROM RollingStats
WHERE ABS((MonthlyTotal - RollingAvg) / NULLIF(RollingStdDev, 0)) > 2.5;
```

Q2: Recursive CTE for Customer Hierarchy

Build a recursive query to traverse organizational customer hierarchies for consolidated billing.

```
WITH RECURSIVE CustomerHierarchy AS (
  SELECT CustomerID, ParentCustomerID, CustomerName, 0 AS Level,
    CAST(CustomerName AS VARCHAR(500)) AS Path
  FROM Customers WHERE ParentCustomerID IS NULL
  UNION ALL
  SELECT c.CustomerID, c.ParentCustomerID, c.CustomerName, ch.Level + 1,
    ch.Path + ' > ' + c.CustomerName
  FROM Customers c
  INNER JOIN CustomerHierarchy ch ON c.ParentCustomerID = ch.CustomerID
)
SELECT * FROM CustomerHierarchy ORDER BY Path;
```

Q3: Performance Optimization for Large Datasets

Optimize a query processing 50M+ meter readings with proper indexing strategy.

Key Optimizations:

- Clustered index on (CustomerID, ReadingDate) for range queries
- Partitioning by year/month for partition elimination
- Filtered indexes for active vs. inactive meters
- Columnstore for analytical workloads

Q4: Data Quality Validation Query

Create a comprehensive data quality report identifying anomalies in meter readings.

```
SELECT 'Negative Consumption' AS IssueType, COUNT(*) AS RecordCount
FROM MeterReadings WHERE Consumption < 0
UNION ALL
SELECT 'Future Reading Date' AS IssueType, COUNT(*)
FROM MeterReadings WHERE ReadingDate > GETDATE()
UNION ALL
SELECT 'Outlier Consumption (>3 std dev)' AS IssueType, COUNT(*)
FROM MeterReadings
WHERE Consumption > (SELECT AVG(Consumption) + 3*STDDEV(Consumption) FROM MeterReadings);
```

Q5: Customer Segmentation Using SQL

Segment customers into quartiles based on consumption patterns for targeted interventions.

```
WITH CustomerMetrics AS (
  SELECT CustomerID, AVG(Consumption) AS AvgConsumption,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY Consumption) AS MedianConsumption
  FROM MeterReadings GROUP BY CustomerID
),
Segmented AS (
  SELECT *, NTILE(4) OVER (ORDER BY AvgConsumption) AS ConsumptionQuartile,
    CASE WHEN AvgConsumption > MedianConsumption * 1.5 THEN 'High Variability'
         WHEN AvgConsumption < MedianConsumption * 0.5 THEN 'Low Usage'
         ELSE 'Standard' END AS UsagePattern
  FROM CustomerMetrics
)
SELECT * FROM Segmented;
```

Q6: Leakage Detection Pattern Query

Identify potential leakage through continuous low-flow patterns overnight.

```
WITH NightReadings AS (
  SELECT CustomerID, ReadingDate, Consumption,
    LAG(Consumption) OVER (PARTITION BY CustomerID ORDER BY ReadingDate) AS PrevConsumption
  FROM MeterReadings WHERE DATEPART(hour, ReadingDate) BETWEEN 0 AND 5
),
LeakageCandidates AS (
  SELECT CustomerID, COUNT(*) AS NightReadingsCount,
    AVG(Consumption - PrevConsumption) AS AvgNightFlow
  FROM NightReadings WHERE Consumption > PrevConsumption
  GROUP BY CustomerID
  HAVING COUNT(*) > 20 AND AVG(Consumption - PrevConsumption) > 0.1
)
SELECT * FROM LeakageCandidates;
```

Q7: Billing Calculation Edge Cases

Handle prorated billing for mid-cycle meter changes and tariff transitions.

```
WITH BillingPeriods AS (
  SELECT CustomerID, BillingStart, BillingEnd,
    DATEDIFF(day, BillingStart, BillingEnd) AS DaysInPeriod,
    TariffRate, OpeningRead, ClosingRead
  FROM BillingCycles
),
ProratedBills AS (
  SELECT CustomerID, (ClosingRead - OpeningRead) * TariffRate *
    (DaysInPeriod / 30.0) AS ProratedAmount
  FROM BillingPeriods
)
SELECT * FROM ProratedBills;
```

Q8: Regulatory Reporting Aggregations

Generate OFWAT-compliant performance reports with required metrics.

Key Metrics:

- Per Capita Consumption (PCC)
- Leakage per km of mains
- Customer minutes lost (supply interruptions)
- Response times to customer contacts

Q9: Slowly Changing Dimension Handling

Implement Type 2 SCD for customer address changes with effective dates.

```
MERGE INTO CustomerHistory AS target
USING (SELECT * FROM CustomerUpdates) AS source
ON target.CustomerID = source.CustomerID AND target.IsCurrent = 1
WHEN MATCHED AND (target.Address <> source.Address) THEN
  UPDATE SET IsCurrent = 0, EndDate = GETDATE()
WHEN NOT MATCHED THEN
  INSERT (CustomerID, Address, StartDate, EndDate, IsCurrent)
  VALUES (source.CustomerID, source.Address, GETDATE(), NULL, 1);
```

Q10: Complex Multi-Table Joins

Join meter readings, customers, tariffs, and weather data for comprehensive analysis.

```
SELECT m.CustomerID, c.CustomerName, t.TariffName,
  m.ReadingDate, m.Consumption, w.Temperature,
  (m.Consumption * t.UnitRate) AS EstimatedCost
FROM MeterReadings m
INNER JOIN Customers c ON m.CustomerID = c.CustomerID
INNER JOIN Tariffs t ON c.TariffID = t.TariffID
LEFT JOIN WeatherData w ON CAST(m.ReadingDate AS DATE) = w.Date
WHERE m.ReadingDate >= '2024-01-01';
```

Q11: SQL Execution Order Trick Question

What is the logical execution order of SQL query clauses?

Answer: FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT

Why this matters: Column aliases defined in SELECT cannot be used in WHERE but CAN be used in ORDER BY.

Q12: Pivot/Unpivot Operations

Transform monthly consumption data from rows to columns for reporting.

```
SELECT CustomerID, [Jan], [Feb], [Mar], [Apr], [May], [Jun],
  [Jul], [Aug], [Sep], [Oct], [Nov], [Dec]
FROM (
  SELECT CustomerID, DATENAME(month, ReadingDate) AS Month, Consumption
  FROM MeterReadings
) AS Source
PIVOT (
  SUM(Consumption) FOR Month IN ([Jan], [Feb], [Mar], [Apr], [May], [Jun],
                                 [Jul], [Aug], [Sep], [Oct], [Nov], [Dec])
) AS PivotTable;
```

Q13: FINAL BOSS - Complete ML Feature Engineering Pipeline

Build a comprehensive feature engineering pipeline for leakage prediction model.

Features to Engineer:

- Rolling statistics (7, 30, 90 day averages)
- Time-based features (hour, day of week, season)
- Lag features (consumption at t-1, t-7, t-30)
- Customer demographics and property characteristics
- Weather correlations
- Anomaly scores using z-scores

Part III: Power BI & Visualization (13 Questions)

Q1: Advanced DAX - Context Transition

Explain context transition and write a DAX measure for revenue-weighted average consumption.

```
Revenue Weighted Avg Consumption =
VAR RevenueByCustomer =
  ADDCOLUMNS(
    VALUES('Dim Customer'[CustomerID]),
    "@CustomerRevenue", [Total Revenue GBP],
    "@RevenueWeight", DIVIDE([Total Revenue GBP], [Total Revenue GBP ALL], 0)
  )
RETURN SUMX(RevenueByCustomer, [@CustomerConsumption] * [@RevenueWeight])
```

Q2: Star Schema Design

Design an optimal star schema for a water utility data warehouse.

Fact Tables:

- FactMeterReadings (grain: individual reading)
- FactBilling (grain: billing period)
- FactCustomerInteractions (grain: interaction)

Dimension Tables:

- DimCustomer (SCD Type 2 for address changes)
- DimDate (standard date dimension)
- DimMeter (meter attributes, installation dates)
- DimTariff (rate structures)
- DimGeography (postcode hierarchy)

Q3: Row-Level Security

Implement RLS for multi-tenant utility dashboards where users only see their region's data.

```
[Region] = USERPRINCIPALNAME() ||
[Region] = LOOKUPVALUE(
  UserRegions[Region],
  UserRegions[Email],
  USERPRINCIPALNAME()
)
```

Q4: Performance Optimization

Optimize a Power BI report handling 50M+ rows of meter data.

Strategies:

- Use Import mode for historical data, DirectQuery for real-time
- Implement aggregations at daily/monthly levels
- Remove unnecessary columns and relationships
- Use composite models with dual storage modes
- Optimize DAX: avoid iterator functions on large tables

Q5: Executive Dashboard Design

Design principles for C-suite water utility dashboards.

Key Principles:

- 5-second rule: Key metrics visible immediately
- Progressive disclosure: Drill from high-level to detail
- Color consistency: Red = bad/problem, Green = good/target met
- Mobile-first: Executives check dashboards on phones
- Action-oriented: Every metric should suggest an action

Q6: Customer Churn Analysis

Build a churn risk visualization with predictive indicators.

```
Churn Risk Score =
VAR AvgConsumption = AVERAGE('FactMeterReadings'[Consumption])
VAR LastReading = MAX('FactMeterReadings'[ReadingDate])
VAR DaysSinceReading = DATEDIFF(LastReading, TODAY(), DAY)
VAR PaymentDelays = CALCULATE(COUNTROWS('FactBilling'), 'FactBilling'[DaysOverdue] > 30)
RETURN
  IF(DaysSinceReading > 60, 30, 0) +
  IF(AvgConsumption < 0.5 * [Historical Avg], 40, 0) +
  IF(PaymentDelays > 2, 30, 0)
```

Q7: Leakage Detection Dashboard

Design dashboard components for leakage monitoring.

Key Visualizations:

- Night flow analysis: Compare 12am-5am consumption patterns
- Minimum night flow trend: Track lowest overnight readings

- Pressure vs flow correlation: Identify pressure-independent flow (leakage signature)
- DMA (District Metered Area) comparison: Benchmark zones

Q8: Regulatory Compliance Reporting

Build OFWAT-compliant performance dashboards.

Required Metrics:

- Service Incentive Mechanism (SIM) scores
- Developer Services outcomes
- Market performance indicators
- Common Performance Commitments (CPCs)

Q9: Time Intelligence in DAX

Calculate year-over-year consumption growth with proper time intelligence.

```
YoY Growth % =
VAR CurrentYear = [Total Consumption]
VAR PriorYear = CALCULATE(
  [Total Consumption],
  SAMEPERIODLASTYEAR('DimDate'[Date])
)
RETURN DIVIDE(CurrentYear - PriorYear, PriorYear, 0)
```

Q10: Bookmarking and Drill-Through

Implement exploratory analysis features for operational users.

Implementation:

- Create bookmark buttons for common filter combinations
- Set up drill-through pages: Customer -> Meter -> Reading Detail
- Use sync slicers for cross-page filtering
- Implement tooltip pages for additional context

Q11: Power BI Service Deployment

Governance and deployment best practices.

Key Considerations:

- Workspace organization: Development, Test, Production
- Dataset refresh scheduling and failure handling
- Version control: Git integration for PBIX files

- Capacity planning: Premium vs Pro licensing decisions

Q12: Python/R Integration

When and how to use Python visuals in Power BI.

Use Cases:

- Advanced statistical visualizations (violin plots, heatmaps)
- Custom forecasting models not available in DAX

Limitations:

- Performance: Python visuals are slower than native
- Interactivity: Limited cross-filtering support
- Deployment: Requires Python runtime on gateway

Q13: Complete CEO Dashboard Design

Design a comprehensive executive dashboard for water utility CEO.

Dashboard Components:

- KPI Ribbon: Customer satisfaction, leakage rate, financial performance
- Trend Analysis: 12-month rolling trends with targets
- OFWAT Performance Matrix: Color-coded compliance status
- Alert Panel: Red/amber/green status indicators
- Drill-through: From summary to detailed operational views

Part IV: Machine Learning & Statistics (13 Questions)

Q1: Time Series Forecasting for Water Demand

Design a forecasting model for water demand prediction.

Key Considerations:

- Multiple seasonalities: Daily, weekly, annual patterns
- External factors: Temperature, rainfall, public holidays
- COVID-19 structural breaks: Model regime changes

```
SARIMAX Model: Phi_P(B^s)phi_p(B)DELTA^dDELTA_s^D y_t = Theta_Q(B^s)theta_q(B)e_t + SUM beta_i
x_{i,t}
```

Q2: Anomaly Detection for Leakage

Build an anomaly detection system for identifying potential leaks.

Approaches:

- Statistical: Z-score, IQR-based outlier detection
- Machine Learning: Isolation Forest, One-Class SVM
- Deep Learning: LSTM autoencoders for time-series

Trade-offs:

- False positive rate vs. detection sensitivity
- Explainability requirements for field teams

Q3: Customer Churn Prediction

Build a churn prediction model for utility customers.

Feature Engineering:

- Recency: Days since last meter reading
- Frequency: Billing/payment patterns
- Monetary: Payment delays, arrears
- Engagement: Customer service contacts

Model Selection:

- Survival analysis: Cox proportional hazards
- Classification: XGBoost with temporal validation

Q4: Model Interpretability (SHAP/LIME)

Explain model predictions for regulatory compliance.

```
Shapley Value Formula: phi_j = SUM_{S subset N\\{j}} |S|!(|N|-|S|-1)!/|N|! [f(S union {j}) -
f(S)]
```

Regulatory Requirements:

- Right to explanation (GDPR Article 22)
- Audit trail for automated decisions
- Non-discrimination verification

Q5: Experimental Design for A/B Testing

Design an experiment to test a new pricing strategy.

Design Elements:

- Randomization: Stratified by customer segment
- Sample size: Power analysis for MDE (Minimum Detectable Effect)
- Duration: Account for billing cycles and seasonality
- Success metrics: Revenue, churn, customer satisfaction

Q6: Bayesian vs Frequentist Approaches

When to use Bayesian methods in utility forecasting.

Bayesian Advantages:

- Incorporate prior knowledge (e.g., seasonal patterns)
- Uncertainty quantification in predictions
- Small sample learning

Frequentist Advantages:

- Computational efficiency
- Established regulatory acceptance

Q7: Bias-Variance Tradeoff

Apply bias-variance decomposition to utility models.

```
MSE = Bias^2 + Variance + Irreducible Error
```

Utility Context:

- High bias: Simple seasonal averages (underfitting)
- High variance: Complex gradient boosting (overfitting to weather noise)
- Optimal: Regularized models with domain constraints

Q8: Feature Engineering for Time Series

Create features for utility demand forecasting.

Temporal Features:

- Lag features: t-1, t-7, t-365 day consumption
- Rolling statistics: 7, 30, 90-day means and std devs
- Seasonal indicators: Month, week of year, holidays

External Features:

- Weather: Temperature, rainfall, daylight hours
- Economic: Regional unemployment, housing starts

Q9: Cross-Validation for Temporal Data

Why standard k-fold CV fails for time series and what to use instead.

Problem:

- Random k-fold leaks future information into training

Solution:

- Time Series Split: Expanding window validation
- Walk-forward validation: Rolling origin
- Blocked CV: Preserve temporal structure

Q10: When NOT to Use ML

Business judgment scenarios where ML is inappropriate.

Avoid ML When:

- Sample size < 1000 (insufficient data)
- Requirements for perfect explainability (use rules instead)
- Problem can be solved with simple heuristics
- Cost of false positives is extremely high
- Regulatory constraints prohibit black-box models

Q11: Optimization Algorithms

Apply optimization for resource allocation in utilities.

Applications:

- Crew scheduling: Minimize response times
- Leakage detection routing: Traveling salesman variants
- Pump scheduling: Minimize energy costs

Techniques:

- Linear programming for convex problems
- Genetic algorithms for complex constraints

Q12: Handling Concept Drift

Detect and adapt to changing patterns in production models.

Types of Drift:

- Sudden: COVID-19 lockdowns, major infrastructure changes
- Gradual: Climate change effects on demand patterns
- Seasonal: Expected recurring changes

Mitigation:

- Online learning: Continuous model updates
- Ensemble methods: Combine multiple time-window models

Q13: End-to-End Prescriptive Analytics

Design a complete system from prediction to action recommendation.

System Components:

- Predictive layer: Leakage probability scoring
- Optimization layer: Prioritize inspections by expected value
- Decision layer: Recommend specific actions with confidence
- Feedback loop: Track outcomes to improve recommendations

Part V: Business Case Studies (13 Questions)

Q1: The GBP 5 Million Leakage Detection Investment

Build a business case for AI-powered leakage detection with 8-12% reduction promise.

Business Case Structure:

- Problem: Current leakage rate, cost per ML, regulatory pressure
- Solution: AI detection system with sensor network
- Benefits: 8-12% leakage reduction = GBP 2-3M annual savings
- Costs: GBP 5M implementation over 2 years
- ROI: 3-year payback, NPV positive at 8% discount rate
- Risks: Technology maturity, integration complexity

Q2: Defining KPIs for Customer Satisfaction

Design metrics for a customer satisfaction initiative.

Leading Indicators:

- First contact resolution rate
- Average speed of answer
- Digital adoption rate

Lagging Indicators:

- Net Promoter Score (NPS)
- Customer effort score
- Complaint volume and resolution time

Q3: ROI Calculation for Predictive Maintenance

Calculate ROI for predictive maintenance program for pump stations.

Calculation Framework:

- Current state: Reactive maintenance costs, unplanned outage frequency
- Investment: Sensors, analytics platform, training
- Benefits: Reduced emergency repairs, extended asset life
- Risk-adjusted: Apply probability weights to scenarios

Q4: Stakeholder Management Challenge

Convince skeptical executives to adopt data-driven decision making.

Strategy:

- Start with quick wins: Low-risk, high-visibility projects
- Speak their language: Translate technical metrics to business outcomes
- Build trust: Transparent about limitations and uncertainties
- Co-create: Involve skeptics in solution design

Q5: Regulatory Compliance Strategy

Develop OFWAT PR24 compliance reporting strategy.

Key Elements:

- Understand requirements: Common Performance Commitments
- Data governance: Audit trails, data lineage
- Validation: Independent verification of metrics
- Continuous improvement: Performance commitment delivery

Q6: Customer Segmentation Strategy

Design segmentation for targeted customer interventions.

Segmentation Dimensions:

- Value: High/medium/low consumption and revenue
- Behavior: Payment patterns, engagement level
- Risk: Churn probability, credit risk
- Needs: Digital preference, service requirements

Q7: Project Prioritization Framework

Prioritize multiple competing data projects with limited resources.

Framework:

- Value: Expected business impact (revenue, cost savings)
- Effort: Implementation complexity and timeline
- Strategic alignment: Fit with organizational priorities
- Risk: Technical and business risk assessment
- Dependencies: Prerequisites and blocker analysis

Q8: Change Management for Dashboard Adoption

Drive adoption of new dashboards across the organization.

ADKAR Framework:

- Awareness: Communicate why change is needed
- Desire: Build motivation through WIIFM (What's In It For Me)
- Knowledge: Training on new tools and processes
- Ability: Hands-on practice with support
- Reinforcement: Celebrate wins, address resistance

Q9: Ethical Dilemma - Uncomfortable Truth

Data reveals a service quality problem. How do you handle it?

Response Framework:

- Verify: Confirm findings with multiple data sources
- Contextualize: Understand root causes and scope
- Escalate: Inform appropriate leadership with recommendations
- Act: Develop remediation plan with clear timeline
- Communicate: Transparent with affected customers

Q10: Resource Allocation Dilemma

Limited budget, multiple opportunities - how do you decide?

Decision Criteria:

- Quick wins: Immediate value with low effort
- Strategic bets: High value, high risk, long-term
- Foundation: Capabilities needed for future projects
- Portfolio balance: Mix of risk profiles and timelines

Q11: Measuring Impact of Prescriptive Analytics

How do you measure the business impact of prescriptive analytics?

Measurement Approach:

- Baseline: Establish pre-implementation performance
- Counterfactual: Estimate what would have happened without intervention
- Attribution: Isolate analytics contribution from other factors

- Longitudinal: Track impact over time

Q12: Conflicting Requirements Navigation

Different departments want opposite things from the same data project.

Resolution Strategy:

- Understand: Deep dive into each department's needs
- Find common ground: Identify shared objectives
- Prioritize: Use objective criteria for trade-offs
- Iterate: Phased delivery addressing both needs over time

Q13: The 90-Day CEO Challenge

Show measurable data science value in first 90 days.

90-Day Plan:

- Days 1-30: Quick wins - automate existing reports, fix data quality issues
- Days 31-60: Deliver first insights - customer segmentation, anomaly detection
- Days 61-90: Show measurable impact - cost savings, efficiency gains

Part VI: Data Architecture (12 Questions)

Q1: Medallion Architecture for Water Utility

Design a modern data platform using medallion architecture.

Architecture Layers:

- Bronze: Raw ingestion from 15M daily meter readings via Event Hubs
- Silver: Cleansed, conformed data with quality checks
- Gold: Business-ready aggregates, customer 360, OFWAT reporting

Q2: ETL vs ELT Decision Framework

When to use ETL vs ELT for different use cases.

Factor	ETL	ELT
Data Volume	Small-Medium	Large
Transformation Complexity	Complex	Simple-Moderate
Target System	Traditional DW	Cloud Data Lake
Use Case	Regulatory reporting	Exploratory analytics

Q3: Data Quality Framework

Design comprehensive data quality monitoring.

Dimensions:

- Completeness: Missing value rates
- Accuracy: Validation against reference data
- Consistency: Cross-system reconciliation
- Timeliness: Data freshness SLAs
- Validity: Format and range checks

Q4: GDPR Compliance Architecture

Implement data lineage and audit trails for GDPR compliance.

Components:

- Data catalog: Automated discovery and classification
- Lineage tracking: End-to-end data flow visualization
- Access logging: Who accessed what when
- Retention policies: Automated data lifecycle management

Q5: Real-Time vs Batch Processing

Design for smart meter data processing at scale.

Current State: 500K meters, quarterly reads

Target State: 2M meters, daily reads (8x volume increase)

Future State: 15-min intervals = 8,760x volume increase

Solution: Lambda architecture with batch and speed layers

Q6: Azure Data Factory Pipeline Design

Orchestrate complex data workflows with ADF.

Pipeline Components:

- Triggers: Schedule, event-based, tumbling window
- Activities: Copy, transform, stored procedure, Databricks
- Control flow: If conditions, loops, switches
- Monitoring: Alerting on failures and SLA breaches

Q7: Slowly Changing Dimensions

Handle changing customer attributes in the data warehouse.

SCD Types:

- Type 0: Fixed, never changes
- Type 1: Overwrite (no history)
- Type 2: Add new row with effective dates
- Type 3: Add previous value column

Q8: Data Governance Framework

Establish enterprise data governance for utilities.

Governance Pillars:

- Data ownership: Clear accountability for each dataset
- Data quality: Standards and monitoring

- Metadata management: Business and technical definitions
- Security and privacy: Access controls and compliance

Q9: Scaling Challenges

Handle growth from thousands to millions of smart meters.

Scaling Strategies:

- Horizontal partitioning: Shard by customer or geography
- Time-based partitioning: Partition by date for efficient queries
- Caching: Hot data in memory, cold in object storage
- Compression: Columnar formats (Parquet) for analytics

Q10: Integration Architecture

Connect operational systems to analytics platform.

Integration Patterns:

- CDC (Change Data Capture): Real-time replication
- API-based: REST/GraphQL for modern systems
- File-based: SFTP for legacy systems
- Message queues: Event-driven async integration

Q11: Disaster Recovery

Ensure business continuity for data platforms.

DR Strategies:

- Backup: Regular snapshots with point-in-time recovery
- Replication: Geo-redundant storage
- Failover: Automated or manual DR site activation
- RTO/RPO: Define recovery time and point objectives

Q12: Cost Optimization

Optimize cloud data platform costs.

Optimization Strategies:

- Right-sizing: Match compute to workload
- Reserved capacity: Pre-purchase for predictable workloads
- Auto-pause: Stop dev/test environments overnight

- Storage tiers: Hot, cool, archive based on access patterns

Part VII: Python Programming (12 Questions)

Q1: Complex Pandas Operations

Multi-index operations and performance optimization.

```
import pandas as pd
import numpy as np

# Multi-index time series
df = df.set_index(['customer_id', 'reading_date'])

# Vectorized rolling statistics
df['rolling_mean_7d'] = df.groupby('customer_id')['consumption']
    .transform(lambda x: x.rolling(7, min_periods=1).mean())

# Avoid iteration - use vectorization
# BAD: for row in df.itertuples(): ...
# GOOD: df['new_col'] = df['col1'] * df['col2']
```

Q2: Data Cleaning Pipeline

Handle messy utility meter data with missing values and outliers.

```
def clean_meter_data(df):
    # Remove duplicates
    df = df.drop_duplicates(subset=['meter_id', 'reading_date'])

    # Handle negative consumption
    df.loc[df['consumption'] < 0, 'consumption'] = np.nan

    # Outlier detection using IQR
    Q1 = df['consumption'].quantile(0.25)
    Q3 = df['consumption'].quantile(0.75)
    IQR = Q3 - Q1
    outlier_mask = (df['consumption'] < Q1 - 1.5*IQR) | (df['consumption'] > Q3 + 1.5*IQR)
    df.loc[outlier_mask, 'consumption'] = np.nan

    return df
```

Q3: Configurable ETL Pipeline

Build production-ready ETL with error handling and logging.

```
import logging
from dataclasses import dataclass

@dataclass
class ETLConfig:
    source_path: str
    destination_table: str
    batch_size: int = 10000

class ETLPipeline:
    def __init__(self, config: ETLConfig):
        self.config = config
        self.logger = logging.getLogger(__name__)

    def run(self):
        try:
            self.logger.info(f'Starting ETL: {self.config.source_path}')
            data = self.extract()
            transformed = self.transform(data)
            self.load(transformed)
        except Exception as e:
            self.logger.error(f'ETL failed: {e}')
            raise
```

Q4: Memory-Efficient Processing

Process millions of readings without loading everything into memory.

```
# Chunked processing
chunk_size = 100000
for chunk in pd.read_csv('meter_readings.csv', chunksize=chunk_size):
    process_chunk(chunk)

# Use categorical types for low-cardinality columns
df['customer_type'] = df['customer_type'].astype('category')

# Downcast numeric types
df['consumption'] = pd.to_numeric(df['consumption'], downcast='float')
```

Q5: FastAPI for Data Services

Design REST API for meter data queries.

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()

@app.get('/meters/{meter_id}/readings')
async def get_readings(meter_id: str, start_date: str, end_date: str, limit: int = 1000):
    readings = query_database(meter_id, start_date, end_date, limit)
    if not readings:
        raise HTTPException(status_code=404, detail='No readings found')
    return {'meter_id': meter_id, 'readings': readings}
```

Q6: Automated Report Generation

Generate formatted reports with visualizations.

```
from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill

def generate_monthly_report(data, output_path):
    wb = Workbook()
    ws = wb.active
    ws.title = 'Monthly Summary'

    # Header styling
    header_fill = PatternFill(start_color='366092', end_color='366092', fill_type='solid')
    for col, header in enumerate(['Customer', 'Consumption', 'Cost'], 1):
        cell = ws.cell(row=1, column=col, value=header)
        cell.font = Font(bold=True, color='FFFFFF')
        cell.fill = header_fill

    wb.save(output_path)
```

Q7: Feature Engineering

Create time-series features for ML models.

```
def create_time_features(df, datetime_col):
    df['hour'] = df[datetime_col].dt.hour
    df['day_of_week'] = df[datetime_col].dt.dayofweek
    df['month'] = df[datetime_col].dt.month
    df['is_weekend'] = df['day_of_week'].isin([5, 6]).astype(int)
    df['is_holiday'] = df[datetime_col].isin(holiday_dates).astype(int)

    # Lag features
    for lag in [1, 7, 30]:
        df[f'consumption_lag_{lag}'] = df.groupby('customer_id')['consumption'].shift(lag)

    return df
```

Q8: Unit Testing and Validation

Ensure production pipeline reliability.

```
import pytest
from pandera import DataFrameSchema, Column, Check

# Define data contract
schema = DataFrameSchema({
    'customer_id': Column(str, nullable=False),
    'consumption': Column(float, Check.greater_than_or_equal_to(0)),
    'reading_date': Column('datetime64[ns]', nullable=False)
})

def test_data_quality(df):
    validated = schema.validate(df)
    assert len(validated) == len(df), 'Data validation failed'

# Unit test
def test_calculate_bill():
    assert calculate_bill(100, 1.5) == 150.0
```

Q9: Parallel Processing

Speed up data transformation with multiprocessing.

```
from multiprocessing import Pool

def process_customer(customer_data):
    # Transform single customer data
    return transform(customer_data)

if __name__ == '__main__':
    customers = df.groupby('customer_id')
    with Pool(processes=4) as pool:
        results = pool.map(process_customer, customers)
```

Q10: Custom Anomaly Detection

Implement statistical anomaly detection algorithm.

```
class ZScoreAnomalyDetector:
    def __init__(self, threshold=3):
        self.threshold = threshold
        self.mean = None
        self.std = None

    def fit(self, data):
        self.mean = np.mean(data)
        self.std = np.std(data)
        return self

    def predict(self, data):
        z_scores = np.abs((data - self.mean) / self.std)
        return z_scores > self.threshold
```

Q11: Pipeline Monitoring

Add monitoring and alerting to data pipelines.

```
from dataclasses import dataclass
from datetime import datetime

@dataclass
class PipelineMetrics:
    records_processed: int
    processing_time: float
    error_count: int
    timestamp: datetime

class PipelineMonitor:
    def record_metrics(self, metrics: PipelineMetrics):
        # Send to monitoring system
        if metrics.error_count > 0:
            self.send_alert(f'Pipeline errors: {metrics.error_count}')
```

Q12: Algorithm Challenge

Efficiently match meter readings to billing periods.

```
from bisect import bisect_right

def match_readings_to_periods(readings, billing_periods):
    period_starts = [p['start'] for p in billing_periods]
    matches = []

    for reading in readings:
        idx = bisect_right(period_starts, reading['date']) - 1
        if idx >= 0:
            matches.append({'reading': reading, 'period': billing_periods[idx]})

    return matches  # O(n log m) vs O(n*m) for naive approach
```

Part VIII: Behavioral & Leadership (12 Questions)

Q1: Customer Focus - Difficult Stakeholder

Question: Tell me about a time when you had to work with a senior stakeholder who fundamentally distrusted data-driven recommendations.

What to Include:

- Situation: Context and stakeholder's concerns
- Action: How you built trust and communicated
- Result: Outcome and relationship improvement

Red Flags to Avoid:

- Blaming the stakeholder for being resistant to change
- Claiming you converted them immediately

Q2: Making Things Happen - Tight Deadline

Question: Describe a time when you had to deliver under a tight deadline with incomplete data.

STAR Framework:

- Situation: Project constraints and timeline pressure
- Task: What you needed to deliver
- Action: How you scoped, prioritized, and executed
- Result: What was delivered and lessons learned

Q3: Improving Service - Driving Change

Question: Tell me about a time you challenged an existing process and drove meaningful change.

Key Elements:

- Identify the problem with data/evidence
- Build coalition for change
- Pilot and demonstrate value
- Scale successful approach

Q4: Accuracy & Decision Making - Unexpected Data

Question: Tell me about a time when data told you something unexpected or unwelcome.

Strong Response:

- Verified findings with multiple sources
- Understood implications before communicating
- Presented with context and recommendations
- Led action to address the issue

Q5: Collaboration - Difficult Team Member

Question: Describe working with a difficult team member on a critical project.

Approach:

- Understand their perspective and concerns
- Find common ground and shared goals
- Establish clear communication norms
- Focus on project success, not personal issues

Q6: Communication - Technical to Non-Technical

Question: Give an example of explaining a complex technical concept to a non-technical executive.

Best Practices:

- Use analogies relevant to their domain
- Focus on business impact, not technical details
- Visual aids and simple language
- Check for understanding and invite questions

Q7: Developing Self - Rapid Learning

Question: Tell me about a time you had to learn a new skill quickly for a business need.

What to Demonstrate:

- Learning methodology and resources used
- How you applied learning in practice
- Outcome and continued development

Q8: Dependable - Taking Ownership

Question: Describe a time when something went wrong and you took ownership.

Strong Response:

- Acknowledged the issue without deflection

- Analyzed root cause systematically
- Implemented fix and preventive measures
- Communicated transparently with stakeholders

Q9: Knowledgeable - Admitting Limitations

Question: Tell me about a time you did not know something and how you handled it.

Good Answer Shows:

- Intellectual honesty and humility
- Resourcefulness in finding answers
- Knowing when to ask for help
- Learning from the experience

Q10: Progressive - Challenging Tradition

Question: Give an example of challenging a traditional approach with an innovative solution.

Structure:

- Traditional approach and its limitations
- Your innovative alternative
- How you gained buy-in
- Results and adoption

Q11: Leadership Without Authority

Question: Describe a time you influenced a decision you did not own.

Key Techniques:

- Build credibility through expertise
- Understand decision-maker's priorities
- Present data-driven recommendations
- Find allies and build consensus

Q12: Ethical Dilemma - Privacy vs Value

Question: The Sales Director wants to use behavioral data customers did not consent to for retention marketing. What do you do?

Response Framework:

- Understand the request and business rationale

- Explain GDPR/legal requirements clearly
- Propose compliant alternatives
- Escalate if pressure continues
- Document the decision process

Part IX: Ambiguous Pressure Tests (12 Questions)

Q1: The CEO's Ambush

Scenario: "Our customer churn is increasing. I need to know why by end of day." - CEO walks away. No data access yet.

Clarifying Questions to Ask:

- What time period are we analyzing?
- How is churn defined? (cancellation, non-payment, etc.)
- What data sources are available?
- What hypotheses should we test first?

Approach:

- Acknowledge urgency while setting realistic expectations
- Prioritize: Quick data pull vs. deep analysis
- Deliver preliminary findings with confidence intervals

Q2: Mystery Dataset

Scenario: You're given a dataset with no documentation. What do you do?

Systematic Approach:

- Profile: Row counts, column types, missing values
- Explore: Distributions, correlations, outliers
- Validate: Check against known reference data
- Document: Create data dictionary as you discover

Q3: Conflicting Executives

Scenario: Two executives want opposite things based on the same data.

Navigation Strategy:

- Understand both perspectives fully
- Identify where interpretations differ
- Present analysis showing both viewpoints
- Facilitate data-driven discussion

Q4: Vague Directive

Scenario: "Fix our data quality issues" - no specifics given.

Response:

- Define scope: Which systems, data domains, timeframes?
- Assess current state: Data quality audit
- Prioritize: Impact vs. effort matrix
- Propose phased approach with quick wins

Q5: Model Bias Crisis

Scenario: A model you deployed is showing unexpected bias. Handle the crisis.

Crisis Response:

- Immediate: Assess severity and potential impact
- Contain: Disable or limit model if necessary
- Investigate: Root cause analysis
- Communicate: Transparent with stakeholders
- Remediate: Fix and revalidate

Q6: Ethical Pressure

Scenario: "Make the data tell the story we want" - ethical tension.

Response:

- Clarify: Understand what they are really asking
- Educate: Explain data integrity principles
- Alternative: Propose honest framing that supports goals
- Escalate: If unethical pressure continues

Q7: Compressed Timeline

Scenario: Project deadline cut in half. How do you deliver value?

Approach:

- Re-scope: Identify MVP vs. nice-to-have
- Prioritize: Highest impact deliverables first
- Communicate: Set expectations on what is feasible
- Iterate: Deliver incrementally

Q8: Algorithm Skepticism

Scenario: Stakeholder dismisses your analysis: "I do not trust algorithms."

Response:

- Acknowledge: Validate their concern
- Explain: Show how the algorithm works simply
- Demonstrate: Compare to their intuition
- Build trust: Start with smaller decisions

Q9: Sensitive Discovery

Scenario: Data reveals a problem that will cause panic if widely known.

Handling:

- Verify: Confirm findings are accurate
- Escalate: Inform appropriate leadership
- Recommend: Propose measured response
- Support: Help develop communication strategy

Q10: Impossible Request

Scenario: Asked to build something you know is technically infeasible.

Response:

- Explain constraints clearly
- Propose alternative approaches
- Show trade-offs of different options
- Collaborate on feasible solution

Q11: Resource Constraints

Scenario: Multiple stakeholders, conflicting priorities, limited resources.

Decision Framework:

- Map all requests to business value
- Assess dependencies and blockers
- Facilitate prioritization workshop
- Produce transparent prioritization matrix

Q12: The Unanswerable

Scenario: "How much revenue will we lose if there is a major water quality incident next year?" - No historical data.

Approach:

- Acknowledge uncertainty
- Use analogies: Similar incidents in other utilities
- Scenario planning: Best/worst/most likely cases
- Range estimate with confidence intervals
- Identify data to collect for better future estimates

Final Notes

Remember: These questions are designed to test not just technical knowledge, but also business acumen, communication skills, and grace under pressure. Practice your responses using the STAR method for behavioral questions, and always be prepared to explain your reasoning process for technical questions.

Good luck with your Business Stream Data Scientist interview!